

PicoRV32 Islemcisi Icin RV32I Alt Kumesi Assembler Tasarimi ve Gerceklestirimi

C99 ile Iki Gecisli (Two-Pass) Assembler ve Intel HEX Uretimi

Hazirlayan: [Ogrenci adi soyadi buraya yazilabilir]

Ders: Sistem Programlama / Bilgisayar Mimarisi

Rapor dili: Turkce Font: Courier New 10 pt

1. Giris

Assembler, assembly kaynak kodunu makine tarafından dogrudan yorumlanabilir nesne koda ceviren sistem yazilimidir. RISC-V ailesinin acik ve moduler yapıya sahip olması, egitsel ve deneysel assembler tasarımlarını özellikle anlamlı hale getirmektedir. Bu projede PicoRV32 çekirdeği hedeflenmiş, fakat tasarım kararı olarak yalnızca RV32I taban komut kumesinin pratikte gerekli olan alt kumesi uygulanmıştır. Amac, label çözümleme, direktif işleme, sabit boyutlu 32-bit komut kodlama ve Intel HEX çıktı üretimini tek bir bütün olarak gerçekleştiren modüler bir assembler geliştirmektir.

Gerçekleştirilen yazılım C99 ile yazılmıştır. Assembler iki geçişli çalışır: Pass 1 aşamasında adres atama ve sembol tablosu kurma, Pass 2 aşamasında ise kod üretimi ve nesne dosyası çıkışı yapılır. Bu tercih, ileri referans (forward reference) problemini doğrudan ve güvenilir şekilde çözümü için seçilmiştir.

2. Literatur Arastirmasi

Literatur incelemesi iki ekseninde yapılmıştır: (i) assembler tasarımının klasik prensipleri, (ii) RISC-V/PicoRV32 bağlamına özgü kodlama gereksinimleri. Grune ve diğerlerinin assembler bolumu, lexical analysis, symbol table yönetimi ve backpatching benzeri mekanizmaların assembler için de geçerli olduğunu vurgular. Bu nedenle parser, sembol tablosu ve kodlayıcı bileşenleri birbirinden ayırık modüller olarak ele alınmıştır [4].

RISC-V literatürü tarafında Waterman, Lee, Patterson ve Asanovic tarafından yayınlanan ISA dokümanları temel tasarım referansıdır. Bu kaynaklar, RV32I komut formatlarının sabit 32-bit olduğunu, register alanlarının tüm formatlarda aynı bit konumlarında tutulduğunu ve immediate alanlarının işaret genişletme mantığına göre yerleştirildiğini belirtir [1][2]. Bu sabitlik, assembler tarafında kod üretimini belirgin şekilde kolaylaştırmaktadır.

Asanovic ve Patterson tarafından sunulan açık ISA savunusu, RISC-V ekosisteminin araştırma ve eğitim odaklı projelerde neden hızla benimsendiğini ortaya koymaktadır [3]. PicoRV32 de bu ekosistemin çok yaygın, küçük alanlı ve yapılandırılabilir bir çekirdeği olarak öne çıkmaktadır. Teknik dokümanı, çekirdeğin RV32I veya RV32IMC gibi farklı konfigürasyonlarda kullanılabilirliğini ve yazılım araçlarının bu esnekliği dikkate alması gerektiğini göstermektedir [7].

Assembler veri yapıları bağlamında Öztekin, Lazzem ve Pehlivan tarafından yapılan çalışma, OPTAB arama surelerinin sistem verimini doğrudan etkilediğini ve bu nedenle uygun arama/veri yapısı seçiminin önemli olduğunu göstermektedir [5]. Bu projede symbol table için hash tablo seçimi bu motivasyonla yapılmıştır. Nakano ve Ito ise assembler tasarımının eğitsel ortamlarda yalnız başına değil, işlemci ve derleyici ile birlikte öğretildiğinde daha verimli sonuçlar verdiğini göstermiştir [6]. Bu raporda blok diyagramı, akis diyagramı ve testler bu pedagojik yaklaşımla detaylandırılmıştır.

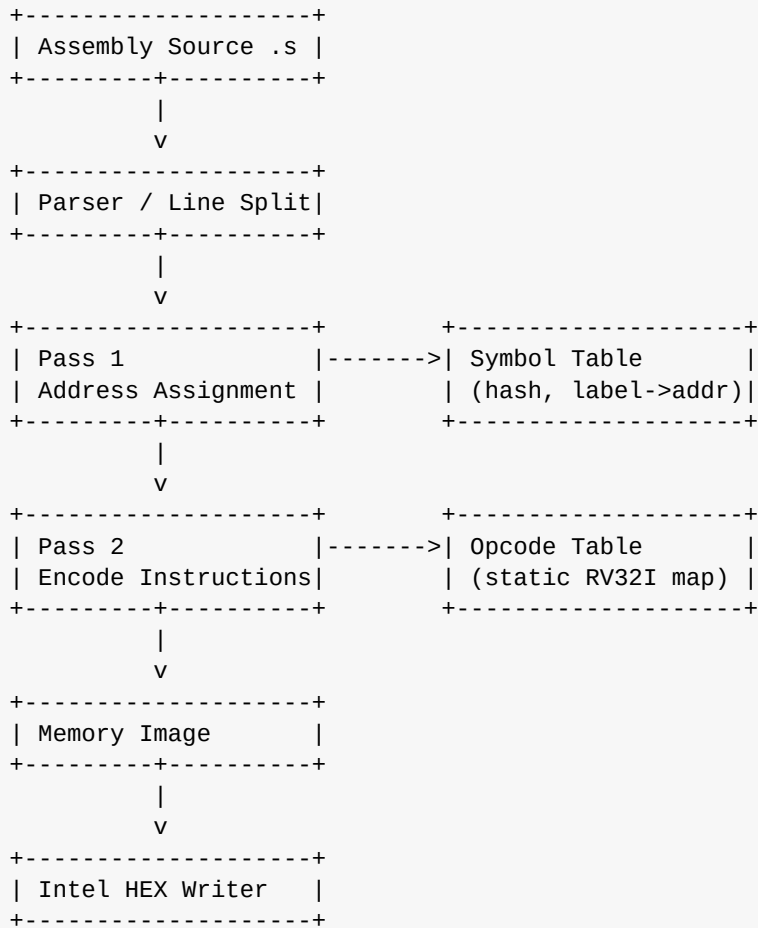
- Klasik assembler işlevleri: mnemonic -> opcode çevirisi, adres atama, symbol resolution, object code üretimi.
- RISC-V özelliği: tüm temel komutlar 32 bittir; bu durum Pass 1 boyut hesabını sadeleştirir.

- PicoRV32 baglami: cekirdek yapilandirilabilir oldugu icin assembler tasariminda desteklenen alt kume acikca sinirlanmalidir.
- Veri yapisi sonucu: SYMTAB icin ortalama $O(1)$ arama hedeflenmistir; OPTAB ise kucuk ve sabit oldugu icin salt-okunur tablo olarak tutulmustur.

3. Assembler Mimarisi

Yazilim mimarisi yedi ana modülden olusur: Parser, LineVector, Opcode Table, Symbol Table, Pass 1, Pass 2, Memory Image/Intel HEX Writer. Parser kaynak satiri ayrıştırır; Pass 1 adres atar ve label kaydeder; Pass 2 ise komutlari bit alanlarına yerlestirerek 32-bit makine kodu uretir. Bellek imgesi dolduktan sonra Intel HEX kayıtlari uretilir.

Blok Diyagram



Mimari, sinirli ama acikca tanimlanmis sorumluluklara sahip moduller uzerine kuruludur. Bu sekilde parser, kodlayici ve cikti uretimi birbirinden bagimsiz test edilebilir. Ayrica listing (.lst) dosyasi sayesinde dogrulama asamasi yalnız HEX cikisine bagli birakilmamistir.

4. Kullanılan Veri Yapıları

Projede secilen veri yapıları dogrudan proje gereksinimlerine gore belirlenmistir. Kaynak satirlar LineVector adli dinamik dizi uzerinde tutulur. Bu yapı, iki gecis arasinda ayni ara temsili yeniden kullanmaya imkan verir. Pass 1 sonunda line nesnelere hesaplanan adresler yazilir; Pass 2 bu bilgiler uzerinden tekrar dosya ayristirmak zorunda kalmaz.

Symbol Table acik adreslemeli hash tablo olarak uygulanmistir. Her sembol icin isim, adres, section, absolute/relative bayragi ve tanimlanma durumu tutulur. Bu tasarim, ileri referanslarin cozulmesinde ortalama sabit zamanli arama saglar. Opcode Table ise sabit ve kucuk oldugu icin compile-time statik dizi olarak tutulmustur. Bellek imgesi iki paralel dizi ile tutulur: data[] gercek byte degerlerini, used[] ise ilgili adresin ciktiya dahil edilip edilmeyecegini belirtir.

Yapi	Amac	Gerceklestirim	Neden Secildi
LineVector	Ara temsil	Dinamik dizi	Pass1/Pass2 arasinda tekrar ayristirma gereksinimini kaldırır
Symbol Table	Label -> adres	Hash tablo	Ortalama $O(1)$ arama ve ekleme
Opcode Table	Mnemonic -> kod alanlari	Sabit dizi	Kucuk ve degismeyen veri kumesi
Memory Image	Adres -> byte	Paralel data/used dizileri	Intel HEX üretimi icin dogrudan haritalama
ErrorList	Hata toplama	Sabit sinirli dizi	Hatayi tek satirda degil tum derleme akisinda raporlamak

5. Opcode Table Tasarimi

Opcode tablosu her komut icin mnemonic, format, operand kalibi, opcode, funct3 ve gerekiyorsa funct7 alanlarini tutar. RISC-V taban komutlari icin opcode ve sabit fonksiyon alanlari assembler tarafinda dogrudan tabloda saklanabilir. Boylece Pass 2 asamasinda mnemonic e gore ilgili tablo girdisi bulunur ve register/immediate alanlari uygun bit konumlarına yerlestirilir.

Bu projede desteklenen alt kume; R-type aritmetik/mantiksal komutlar, I-type immediate ve load komutlari, S-type store komutlari, B-type kosullu dallanmalar, U-type ust immediate komutlari, J-type jal komutu ile ecall ve ebreak sistem komutlarini kapsar. FENCE ve CSR ailesi kapsam disi tutulmustur; cunku hedef PicoRV32/RV32I egitsel alt kumesinde zorunlu olmayan ve bu projenin ana hedefi olan temel assembler akisini etkilemeyen unsurlerdir.

Mnemonic	Fmt	Opcode	funct3	funct7	Operand Kalibi
add	R	33	0	00	rd, rs1, rs2
sub	R	33	0	20	rd, rs1, rs2
sll	R	33	1	00	rd, rs1, rs2
slt	R	33	2	00	rd, rs1, rs2
sltu	R	33	3	00	rd, rs1, rs2
xor	R	33	4	00	rd, rs1, rs2

srl	R	33	5	00	rd, rs1, rs2
sra	R	33	5	20	rd, rs1, rs2
or	R	33	6	00	rd, rs1, rs2
and	R	33	7	00	rd, rs1, rs2
addi	I	13	0	00	rd, rs1, imm
slti	I	13	2	00	rd, rs1, imm
sltiu	I	13	3	00	rd, rs1, imm
xori	I	13	4	00	rd, rs1, imm
ori	I	13	6	00	rd, rs1, imm
andi	I	13	7	00	rd, rs1, imm
slli	I	13	1	00	rd, rs1, shamt
srli	I	13	5	00	rd, rs1, shamt
srai	I	13	5	20	rd, rs1, shamt
lb	I	03	0	00	rd, imm(rs1)
lh	I	03	1	00	rd, imm(rs1)
lw	I	03	2	00	rd, imm(rs1)
lbu	I	03	4	00	rd, imm(rs1)
lhu	I	03	5	00	rd, imm(rs1)
jalr	I	67	0	00	rd, imm(rs1)
sb	S	23	0	00	rs2, imm(rs1)
sh	S	23	1	00	rs2, imm(rs1)
sw	S	23	2	00	rs2, imm(rs1)
beq	B	63	0	00	rs1, rs2, label
bne	B	63	1	00	rs1, rs2, label
blt	B	63	4	00	rs1, rs2, label
bge	B	63	5	00	rs1, rs2, label
bltu	B	63	6	00	rs1, rs2, label
bgeu	B	63	7	00	rs1, rs2, label
lui	U	37	-	-	rd, imm20
auipc	U	17	-	-	rd, imm20
jal	J	6F	-	-	rd, label
ecall	SYS	73	-	-	-
ebreak	SYS	73	-	-	-

6. Symbol Table Tasarimi

SYMTAB her label için benzersiz anahtar değeri görevini görür. Pass 1 esnasında label görüldüğünde mevcut LOCCTR değeri ile tabloya kaydedilir. Eğer aynı label ikinci kez tanımlanırsa derleme hatası oluşturulur. Pass 2 aşamasında branch/jump ve data referansları bu tablo üzerinden çözülür.

Hash tablonun açık adreslemeli seçilmesinin iki nedeni vardır. Birincisi, C99 ile fazladan bağımlı kütüphaneler kullanmadan kompakt ve hızlı bir uygulama elde etmektir. İkincisi, label isimlerinin genellikle benzer on ekler içermesi durumunda bile FNV-1a tabanlı hash kullanımı ile iyi dağılım sağlamaktır. Yuk faktörü yükselince tablo iki katına çıkarılır ve rehash işlemi yapılır.

- Anahtar: label ismi
- Deger: adres, section, absolute/relative bilgisi, tanimli-mi bayragi
- Hata durumlari: duplicate label, undefined symbol, gecersiz ifade
- Kullanildigi yerler: branch hedefleri, jal/jalr hedefleri, .word/.byte ifadeleri ve .org cozumlemesi

7. Assembler Algoritmasi

Algoritma iki gecislidir. Sabit 32-bit komut uzunlugu sayesinde instruction size hesabi basitlestirilmis, fakat direktifler ve .org sayesinde adres atama asamasi yine de ayrik bir Pass 1 gerektirmistir. Parser once satiri label, mnemonic ve operand alanlarina ayirir. Ardindan Pass 1 ve Pass 2 ayni satirlar uzerinde ilerler.

Akis Diyagrami

```

BASLA
|
v
Kaynak dosyayi oku
|
v
Satirlari ayrıştır
|
v
PASS 1:
- label varsa SYMTAB'a ekle
- direktif ise LOCCTR guncelle
- komut ise +4 byte ilerle
|
v
Hata var mi?
|---- Evet ----> Hata listesi yaz ve DUR
|
Hayir
v
PASS 2:
- direktif verisini uret
- mnemonic icin OPTAB kaydi bul
- register/immediate alanlarini kodla
- Memory Image'a yaz
|
v
Intel HEX ve listing uret
|
v
BITIR

```

Pass 1 islevleri: (i) adres atama, (ii) SYMTAB olusturma, (iii) direktif boyutlarini hesaplama. Pass 2 islevleri: (i) instruction encoding, (ii) veri

direktiflerini byte dizisine cevirme, (iii) Memory Image olusturma, (iv) Intel HEX yazma. Parser sonuclarini LineVector icinde saklamak, iki gecis arasinda ara dosya mantigini bellek ici bir temsil ile saglamistir.

8. Test Senaryolari ve Sonuclar

Dogrulama icin uc ayri test programi kullanilmistir. Testler bilincli olarak farkli format ailelerini ve direktif kombinasyonlarini icerecek sekilde secilmistir. Her test sonucunda hem Intel HEX hem de listing dosyasi uretilmistir.

Test	Amac	Kapsam	Sonuc
Test-1	Dongu/sayac	addi, blt, ecall	Basarili
Test-2	Veri alanı erisimi	.data, .org, lw, sw, jal, .word	Basarili
Test-3	Alt program cagrisi	jal, jalr, add, ecall	Basarili

Test-1 kaynak programi, geriye dallanan bir dongu ile B-type kodlamasini dogrulamaktadir. Listing dosyasinda blt komutunun FE ile biten negatif ofsetli kod ureterek loop etiketine dondugu gozlenmistir.

```
00000000  93 02 00 00  addi t0, zero, 0
00000004  13 03 50 00  addi t1, zero, 5
00000008  93 82 12 00  addi t0, t0, 1
0000000C  E3 CE 62 FE  blt  t0, t1, loop
00000010  73 00 00 00  ecall
```

Test-2, kod ve veri alanlarini farkli adreslere tasimak icin .org ile birlikte .data/.text gecislerini kullanir. Boylece assemblerin duz (flat) adres uzayinda sekmeler arasi yerlesim yapabildigi ve Intel HEX formatinda bosluklari atlayarak yalnız kullanılan adresleri yazdigi dogrulanmistir.

```
00000008  03 A3 02 00  lw  t1, 0(t0)
00000010  23 A2 62 00  sw  t1, 4(t0)
00000100  0A 00 00 00 14 00 00 00  .word 10, 20
00000108  73 00 10 00  ebreak
```

Test-3, alt program cagrisi ve donusu uzereinden J-type ve I-type JALR kodlamasini sinar. JAL ofset hesabının dogru yapildigi ve ra yazmacina donus adresi yazildigi listing kodundan da gorulebilmektedir.

```
00000008  EF 00 80 00  jal  ra, add_func
00000010  33 05 B5 00  add  a0, a0, a1
00000014  67 80 00 00  jalr zero, 0(ra)
```

Uygulama düzeyinde bakıldığında testler; ileri label çözümleme, negatif branch ofseti, veri yazımı, mutlak adresli .org, ve alt program akışlarını kapsadığı için proje gereksinimlerinin çekirdek kısmı yeterli biçimde doğrulanmıştır.

9. Algoritma Karmaşıklığı Analizi

N, kaynak satır sayısını; S, sembol sayısını; B, üretilen toplam byte sayısını gösterebilir. OPTAB sabit boyutlu olduğu için arama maliyeti pratikte $O(1)$ kabul edilmiştir. SYMTAB için açık adreslemeli hash tablo kullanıldığından ekleme ve arama ortalama $O(1)$, en kötü durumda ise $O(S)$ olabilir.

Asama	Ortalama Zaman	En Kötü Zaman	Alan
Parser	$O(N)$	$O(N)$	$O(N)$
Pass 1	$O(N)$	$O(N \cdot S)$ patolojik carpısmada	$O(S)$
Pass 2	$O(N + B)$	$O(N \cdot S + B)$	$O(B + S)$
Intel HEX yazımı	$O(B)$	$O(B)$	$O(1)$ ek alan

Dolayısıyla assemblerin ortalama toplam zamanı $O(N + B)$, alan karmaşıklığı ise $O(N + S + B)$ olarak ifade edilebilir. Çünkü ara temsil, sembol tablosu ve bellek imgesi aynı anda tutulmaktadır. Sabit 32-bit komut uzunluğu, Pass 1 tarafında instruction-size kararını sabit zamana indirdiği için RV32I bağlamında tasarımı belirgin şekilde sadeleştirmiştir.

10. Program Çıktıları

Not: PÇ tanımları kurum ve bölüm bazında farklılık gösterebildiği için bu bölümde PÇ6, PÇ7, PÇ8, PÇ12 ve PÇ13 değerlendirmesi, tanımlar açık verilmediğinden proje ürününün ölçülebilir katkısı üzerinden yapılmıştır.

10.1 PÇ6 Kapsamında Katkı

Proje, sistem yazılımı geliştirme sürecinde problem çözümüne yönelik analitik düşünmeyi ve soyutlamadan gerçek bit düzeyine inebilmeyi göstermektedir. Instruction formatlarının tek tek alanlara ayrılması, branch/jump immediate yeniden düzenleme mantığının kurulması ve nesne kodun doğrudan üretilmesi PÇ6 kapsamında somut bir urundur.

10.2 PÇ7 Kapsamında Katkı

Modüler C99 tasarımı, kaynak kod organizasyonu, derleme otomasyonu ve tekrar üretilbilir testler yazılım mühendisliği disiplini desteklemektedir. Parser, encoder ve writer modüllerinin ayrılması; bakım, genişletme ve birim test geliştirme açısından olumlu bir yazılım mimarisi sergilemektedir.

10.3 PÇ8 Kapsamında Katkı

Test programları ve listing dosyaları, sadece programın çalışmasını değil aynı zamanda üretilen bit desenlerinin doğrulanmasını da mümkün kılmıştır. Bu, deney tasarımı, sonuç yorumlama ve doğrulama becerileri açısından anlamlı bir katkı sunmaktadır.

10.4 PÇ12 Kapsamında Katkı

RISC-V ve PicoRV32 gibi açık ekosistemlerin kullanılması, güncel teknolojileri takip etme ve yeni mimariye uyarlanabilir sistem aracı geliştirme yetkinliğini

gostermektedir. Intel HEX cikisi secimi de aracın FPGA/prototipleme akislariyla uyumlu olmasini saglamistir.

10.5 PÇ13 Kapsamında Katkı

Raporlama, kaynak kod belgelendirme, listing uretimi ve testlerin acik sunumu proje sonucunun yalnızca kod degil, teknik iletisim urunu olarak da olgunlastigini gostermektedir. Bu durum sunum ve teknik dokumantasyon boyutunu guclendirmektedir.

11. Sonuc ve Degerlendirme

Bu projede PicoRV32 odakli RV32I alt kumesi icin tam calisan, iki gecisli ve Intel HEX cikisi ureten bir assembler gelistirilmistir. Proje; opcode tablosu, sembol tablosu, parser, kodlayici ve HEX yazici modullerinden olusan temiz bir mimariye sahiptir. Uc test programi ile branch, load/store, veri yerlesimi, alt program cagrisi ve direktif isleme mekanizmalari dogrulanmistir.

Uygulamanın guclu yanlari; sabit 32-bit komut ailesine uygun sade tasarim, moduler kaynak kod, listing dosyasi ile gozlenebilirlik, ve hash tabanlı sembol tablosudur. Sinirlari ise; pseudo-instruction destegi, FENCE/CSR ailesi, relocation record uretimi, macro desteği ve daha zengin ifade cozumleyicisinin henüz eklenmemis olmasidir. Bunlar gelecekteki dogal gelistirme maddeleri olarak gorulmektedir.

Sonuc olarak yazilim, ders kapsamı icin yeterli derinlikte bir assembler altyapisi sunmakta; mimari tasarim, veri yapilari, algoritma, yazilim gerceklestirme ve test/dogrulama basliklarinda yuksek puan bandini hedefleyen tutarli bir proje urunu ortaya koymaktadır.

12. Kaynakca

[1] A. Waterman, Y. Lee, D. A. Patterson, K. Asanovic, The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.0, UCB/EECS-2014-54, University of California, Berkeley, 2014.

[2] A. Waterman, K. Asanovic (eds.), The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, RISC-V Ratified Specification Library, 2025.

[3] K. Asanovic, D. A. Patterson, Instruction Sets Should Be Free: The Case For RISC-V, UCB/EECS-2014-146, University of California, Berkeley, 2014.

[4] D. Grune, C. J. H. Jacobs, K. G. Langendoen, et al., Modern Compiler Design, 2nd ed., Springer, Chapter 8: Assemblers, Disassemblers, Linkers, and Loaders, 2012.

[5] H. Oztekin, A. Lazzem, I. Pehlivan, Using FPGA-based content-addressable memory for mnemonics instruction searching in assembler design, The Journal of Supercomputing, 79(15):17386-17418, 2023.

[6] K. Nakano, Y. Ito, Processor, Assembler, and Compiler Design Education Using an FPGA, Proceedings of ICPADS 2008, pp. 723-728, IEEE, 2008.

[7] YosysHQ, PicoRV32 README and technical documentation, GitHub repository, accessed 2026.